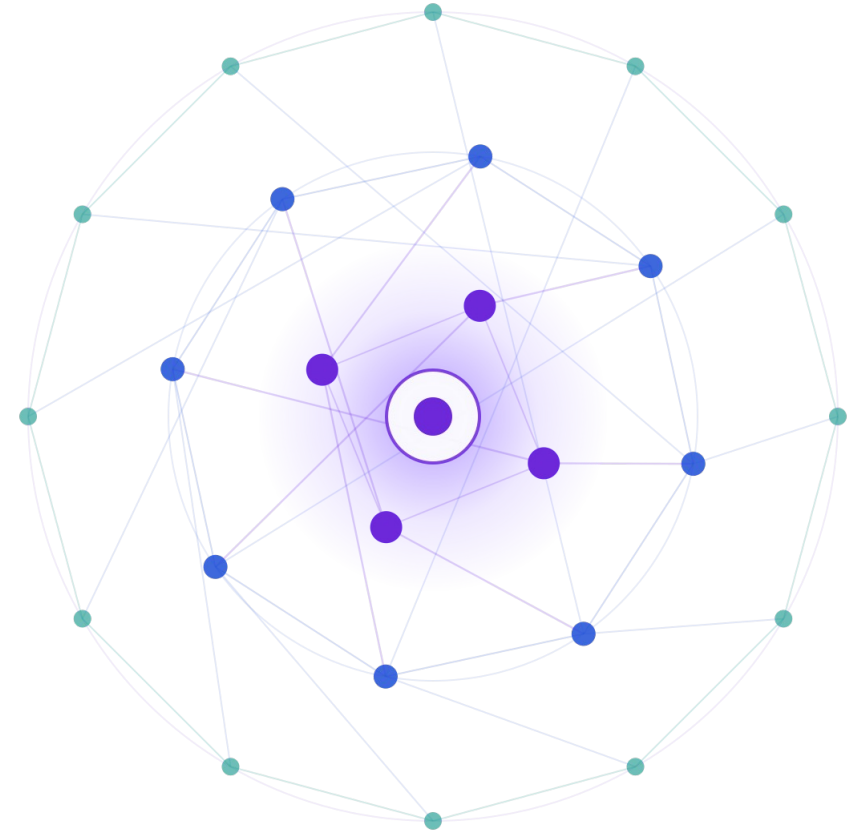


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - Theme 1 — Agentic Code Intelligence
- Team Name - MSRIT_Waypoint
- College Name - M. S. Ramaiah Institute of Technology (MSRIT), Bengaluru
- Member Name & Email 1- Amogh R B — 1ms24ci014@msrit.edu
- Member Name & Email 2- Pankaj Kumar B S — 1ms24ci029@msrit.edu
- Member Name & Email 3- —
- Member Name & Email 4- —
- Submission Github link - <https://github.com/AmoghRB/samsung-prism-genai-2026>



Theme

- **Theme 1 — Agentic Code Intelligence (semantic code retrieval)**
- Given a natural-language query and a library of Python code, return code snippets ranked by relevance.
- **Scored on:** NDCG@10 and MRR on the CoIR apps test split (MTEB AppsRetrieval) — 3,765 queries, 8,765 programs.
- **P1:** retrieval across code versions, with fast index rebuilds.
- **Bonus:** evolutionary retrieval across all versions at once.
- **Constraint:** CPU-only, efficiency (runtime, model size) is scored.

Existing Solutions & Gaps

- **Keyword search (grep, BM25):** misses paraphrase — 'longest rising run' $\neq$ 'LIS'. Queries and code use different vocabularies.
- **LLM ranking:** thousands of long snippets don't fit in a context window, and it is far too slow to sit before generation in a RAG pipeline.
- **General text embedders:** weak on code; we measured gte-modernbert, granite-r2 and others well below code-specific models.
- **Large code embedders (7B+):** accurate but need a GPU — violates the CPU-only constraint.
- **Versioned code:** naive indexes re-embed everything on every commit, and cross-version search returns 5 near-copies of one function.

Our Solutions & Architecture Diagram

- **Pipeline:** query → pre-process → code-embedding (CPU) → exact cosine top-k → ranked snippets
 - Query pre-processing: LaTeX → plain maths ($\$1 \leq n\$ \rightarrow 1 \leq n$), whitespace/markup cleanup, so the 512-token budget carries meaning
 - Embedding: jina-code-embeddings-0.5b with task instruction prompts for queries and code; longest-first batching on CPU
 - Search: exact chunked matrix top-k — milliseconds at this scale, no ANN error
- **Real repositories:** Python ast splits files into functions/classes so hits are the answering block, not a 2,000-line file.
- **Versioned index (P1):** content-addressed — each vector stored under SHA-1 of its code; a version is a manifest. New version ⇒ only changed snippets embedded.
- **Evolutionary search (Bonus):** rank lineages (path::qualname), return best version + which versions share identical code.

Demo & Product Walkthrough

- **Live queries on the APPS corpus:** `python -m src.query --apps -i`
 - Type a problem statement → top-5 Python programs with scores and latency
- **Versioning demo:** `bash examples/versions_demo.sh`
 - Indexes requests v2.28.0, then v2.31.0 and v2.32.3 incrementally
 - Same query: latest version → pinned to v2.28.0 → across all versions
- **Reproduce the score:** `python -m src.evaluate` → `appsretrieval_results.json`
- **Demo video:** <https://drive.google.com/drive/folders/1OfFzQTW2Os6dTVjTHRL54IEJbnADayVx?usp=sharing>

Tools and tech stack used

- **Language:** Python 3.12
- **Model:** jinaai/jina-code-embeddings-0.5b (Hugging Face)
- **Libraries:** sentence-transformers, PyTorch (CPU wheels), NumPy, transformers
- **Evaluation:** MTEB 2.21 — AppsRetrieval task, CoIR apps dataset
- **Code parsing:** Python ast (function/class chunking)
- **Packaging:** Docker (CPU-only image, weights baked in), pytest
- **AI assistance:** Claude Code — disclosed in the AI Disclosure form

Impact & Use case

- **Developer onboarding:** ask 'where is input validated?' in an unfamiliar codebase and land on the function.
- **RAG over code:** fast, CPU-only first stage for code assistants — retrieval runs on any laptop or server, no GPU bill.
- **Code review & debugging:** find how a behaviour was implemented in an older release (version-pinned search).
- **Regression hunting:** cross-version search shows which releases share or changed a function.
- **On-device / enterprise:** 0.5B model, code never leaves the machine.

Innovation highlights, results and limitations

- **Results:** NDCG@10 81.29 · MRR@10 78.02 · Recall@100 97.5 on AppsRetrieval test (full split)
- **Efficiency:** 494M params, CPU-only; live query ~110–180 ms on CPU
- **Innovation:** content-addressed versioned index; lineage-level ranking for evolutionary retrieval; measured query pre-processing
- **Measured, not guessed:** 7 embedding models and 4 query modes swept on the full test split; Matryoshka 256-d keeps 80.1 NDCG at 3.5× less memory
- **Limitations:** single-stage dense retrieval (no reranker — kept out for CPU budget); 512-token truncation of very long programs; bulk indexing of 8.7k programs is slow on laptop CPU (queries are fast)

What's next

- **CPU-budget reranker:** small cross-encoder over top-20 only, measured against latency
- **Quantisation:** int8 / bf16 CPU inference to cut indexing time
- **Hybrid lexical + dense:** catch exact identifiers in real repos
- **Watch mode:** re-index on git commit hooks automatically
- **More languages:** extend the ast chunker beyond Python

Brownie points slide(differentiation)

- **Everything is measured:** every design choice has an NDCG delta in the repo (scratch/results.jsonl), including the ones we rejected.
- **Efficiency-first:** one 0.5B model, no GPU, no reranker — accuracy weighed against runtime and size as FAQ Q31 asks.
- **P1 and Bonus both working, not slides:** incremental re-indexing and lineage-aware cross-version search, with a runnable demo on a real library.
- **Reproducible:** one command reproduces the submitted numbers; Docker image included.

Checklist- Updated on Public GitHub

- Working prototype code — public GitHub repo: Y
- README with reproducible setup instructions: Y
- [Demo video, max 5 minutes: https://drive.google.com/drive/folders/1OfFzQTW2Os6dTVjTHR_L54IEJbnADayVx?usp=sharing](https://drive.google.com/drive/folders/1OfFzQTW2Os6dTVjTHR_L54IEJbnADayVx?usp=sharing)
- Presentation file (PPT): Y
- Repo: <https://github.com/AmoghRB/samsung-prism-genai-2026> · Release tag: PRISM_GENAI_HACKATHON_Y2026

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India